Übungsblatt 7: CPU-Architekturen

Mikrocomputertechnik 3 · 56 Orientierungspunkte · ca. 100–120 Minuten
Eigenarbeit

Musterlösung

Aufgabe 1: Eine sechsstufige Pipeline (4 Punkte)

Papieraufgabe. Eine skalare Pipeline hat die Stufen F, D1, D2, EX, MEM und WB, jeweils mit einem Takt Dauer. Höchstens ein neuer Befehl beginnt je Takt in F; I1 beginnt in Takt 1. Das betrachtete Fenster reicht vom ersten F bis einschließlich des letzten WB. Alle Angaben sind Lehrannahmen, keine Messwerte eines konkreten Kerns.

1. Sieben unabhängige Befehle werden ausgeführt. Bestimmen Sie Fensterlänge, CPI und IPC. Erklären Sie, weshalb die IPC in diesem Fenster kleiner als eins ist. (2 P)
2. Jetzt liest I2 das Ergebnis von I1 in D2. Es gibt kein Forwarding; Schreiben in WB erfolgt vor dem Registerlesen im selben Takt. Ein wartender Leser bleibt in D2 und hält alle jüngeren Befehle auf. Alle übrigen Befehle sind unabhängig. Wann beginnt I2 mit EX? Bestimmen Sie das neue Fenster und dessen CPI. (2 P)

Musterlösung 1

1. I1 belegt die Takte 1 bis 6, I7 die Takte 7 bis 12. Damit sind es $7 + 6 - 1 = 12$ Takte, $\mathrm{CPI} = 12/7 \approx 1{,}714$ und $\mathrm{IPC} = 7/12 \approx 0{,}583$. Das endliche Fenster zählt Füllen und Leeren der Pipeline mit; eine Startkapazität von eins ist keine Zusage für die mittlere bestätigte Rate. **Bewertung:** Fenster 1 P, Kennzahlen und Erklärung 1 P.
2. I1 erreicht WB in Takt 6. I2 erreicht D2 zunächst in Takt 4, kann aber erst in Takt 6 lesen. Seine D2-Phase wird um zwei Takte verlängert, EX beginnt in Takt 7 statt in Takt 5. Die jüngeren Befehle verschieben sich ebenfalls um zwei Takte: Fenster 14, $\mathrm{CPI} = 14/7 = 2$. **Bewertung:** Lesen/EX und zwei Wartetakte 1 P, Fenster/CPI 1 P.

Aufgabe 2: Zwei ALUs sind eine Entwurfsentscheidung (10 Punkte)

Papieraufgabe, M2. Acht RV32I-Register enthalten bereits die Werte $a0, \ldots, a7 = (3, 1, 4, 1, 5, 9, 2, 6)$. Zwei Programme sollen deren Summe liefern. Alle Operationen sind 32-Bit-Additionen modulo 2^{32}.

Befehl	K: eine Kette	Z: zwei verschränkte Ketten
I1	`add t0, a0, a1`	`add t0, a0, a1`
I2	`add t0, t0, a2`	`add t1, a2, a3`
I3	`add t0, t0, a3`	`add t0, t0, a4`
I4	`add t0, t0, a4`	`add t1, t1, a5`
I5	`add t0, t0, a5`	`add t0, t0, a6`
I6	`add t0, t0, a6`	`add t1, t1, a7`
I7	`add t0, t0, a7`	`add t2, t0, t1`

Zeitvertrag: Die Befehle sind vor Takt 1 dekodiert und gegebenenfalls umbenannt. Jede Addition startet am Taktbeginn und liefert ihr Ergebnis am Taktende. Ein Nutzer darf frühestens im Folgetakt starten. Der Start erfolgt in Programmreihenfolge: Ist der nächste Befehl nicht bereit, startet in diesem Takt auch kein jüngerer. Innerhalb eines Takts entsteht kein nutzbares Ergebnis. Retire bestätigt in Programmreihenfolge, frühestens im Takt nach der Fertigstellung. Am Taktbeginn erfolgt zuerst Retire, dann die Auswahl neuer Starts. Das Fenster reicht vom ersten Start bis einschließlich des letzten Retire.

Kern	ALUs	Starts je Takt, höchstens	Retires je Takt, höchstens
D	2	2	2
D1	2	2	1
V	4	4	4

1. Geben Sie für K und Z die RAW-Ketten und das Ergebnisregister samt Wert an. Kann Renaming allein die lange Wertkette von K verkürzen? Begründen Sie. (2 P)
2. Erstellen Sie für D eine Tabelle mit Start- und Retire-Takt von I1 bis I7, jeweils für K und Z. Bestimmen Sie den Speedup von Z gegenüber K. Für Z auf D1 genügt die Retire-Folge mit einer Engpassbegründung. (3 P)
3. Entwerfen Sie eine dritte Variante B mit sieben Additionen als ausgeglichenen Summenbaum. Geben Sie den Code an. Vergleichen Sie auf V die Fenster von B und Z: Weshalb hilft eine weitere Recheneinheit nicht automatisch beiden? (2 P)
4. Für einen langen, unabhängigen Befehlsstrom betrachten Sie nun zusätzlich das Frontend von D: Fetch liefert höchstens 4 Byte je Takt; Decode, Start und Retire schaffen jeweils höchstens zwei Befehle je Takt. Bestimmen Sie die obere Schranke der dauerhaften IPC für nur 32-Bit-Befehle, nur 16-Bit-Befehle und eine mittlere Befehlslänge von 3 Byte. Die kurzen Befehle sind eine abstrakte Modellannahme; eine konkrete komprimierte Übersetzung von K oder Z wird nicht vorausgesetzt. Widersprechen diese Schranken dem kurzen Fenster aus 2? (3 P)

Musterlösung 2

1. K hat die RAW-Kette I1–I2–I3–I4–I5–I6–I7 und liefert `t0 = 31`. Z hat die Ketten I1–I3–I5–I7 und I2–I4–I6–I7; `t2 = 31`. Die längsten Wertketten haben sieben beziehungsweise vier Additionen. Renaming beseitigt Namenskonflikte, erzeugt aber nicht das noch fehlende Ergebnis eines Vorgängers. **Bewertung:** Ketten/Ergebnisse 1 P, Begründung 1 P.

Befehl	D, K: Start / Retire	D, Z: Start / Retire	D1, Z: Retire
I1	1 / 2	1 / 2	2
I2	2 / 3	1 / 2	3
I3	3 / 4	2 / 3	4
I4	4 / 5	2 / 3	5
I5	5 / 6	3 / 4	6
I6	6 / 7	3 / 4	7
I7	7 / 8	4 / 5	8

2. Auf D sinkt das Fenster von acht auf fünf Takte: $S = 8/5 = 1{,}6$. Die IPC steigt von 7/8 auf 7/5. D1 kann die gleichen Operationen genauso früh starten wie D; sein Retire-Pfad bestätigt jedoch erst in den Takten 2 bis 8. Das Fenster bleibt acht Takte, obwohl schon in Takt 4 alles fertig ist. **Bewertung:** D-Tabellen 1 P, Speedup 1 P, D1 und Engpass 1 P.

3. Ein möglicher Baum lautet:

```
add t0, a0, a1    # I1
add t1, a2, a3    # I2
add t2, a4, a5    # I3
add t3, a6, a7    # I4
add t4, t0, t1    # I5
add t5, t2, t3    # I6
add t6, t4, t5    # I7
```

Auf V starten I1–I4 in Takt 1, I5–I6 in Takt 2, I7 in Takt 3; die Retires liegen jeweils einen Takt später. Fenster: vier Takte, Ergebnis `t6 = 31`. Z besitzt trotz vier ALUs weiterhin nur zwei unabhängige Ketten und benötigt bis zum letzten Retire fünf Takte. Die kritische Kette und die Ressourcen sind verschiedene Grenzen. **Bewertung:** korrekter Baum 1 P, Vergleich/Begründung 1 P.

4. Die dauerhafte Rate ist durch $\mathrm{IPC} \le \min(4\,\mathrm{Byte/Takt}/\ell, 2)$ begrenzt. Für $\ell = 4$, 2 beziehungsweise 3 Byte/Befehl folgen $\mathrm{IPC} \le 1$, $\mathrm{IPC} \le 2$ beziehungsweise $\mathrm{IPC} \le 4/3$. Das sind obere Schranken bei ausreichend unabhängiger Arbeit, ohne weitere Engpässe; sie versprechen keine tatsächlich erreichte Rate. Teil 2 startet mit bereits vorbereiteten Befehlen und zählt deren Fetch-Zeit nicht mit. Sein endliches Fenster darf deshalb nicht mit dauerhaftem Nachliefern gleichgesetzt werden. **Bewertung:** drei Schranken 2 P, Unterschied der Fenster 1 P.

Aufgabe 3: Zwei Loads, drei Ausführungsentwürfe (14 Punkte)

Papieraufgabe, Erweiterung von M3. `s0` zeigt auf zwei gültige, ausgerichtete 32-Bit-Wörter. Anfangs gilt:

Speicher / Register	Wert	Speicher / Register	Wert
Wort bei `s0 + 0`	20	Wort bei `s0 + 4`	12
`t0`	60	`t1`	61
`t2`	3	`t3`	70
`t4`	9	`t5`	71
`t6`	72		

```
lw  t0, 0(s0)      # A
add t1, t0, t2     # B
lw  t0, 4(s0)      # C
sub t3, t0, t4     # D
xor t5, t2, t4     # E
and t6, t5, t1     # F
```

Zeit- und Auswahlvertrag für alle drei Entwürfe: Alle sechs Befehle sind vor Takt 1 dekodiert, umbenannt und im ROB aufgenommen. Der Reorder Buffer (ROB) hält die Programmreihenfolge für Retire fest. Höchstens ein Befehl startet je Takt; eine ALU und eine Load/Store-Einheit (LSU) sind vorhanden. Ein Load mit Start in Takt t wird am Ende von $t+4$ fertig; eine ALU-Operation am Ende ihres Starttakts. Nutzer starten frühestens im Folgetakt der Fertigstellung. Am Taktbeginn erfolgen zunächst bis zu zwei Retires in Programmreihenfolge, frühestens im Folgetakt der Fertigstellung, danach die Auswahl eines neuen Starts. ROB und physische Register sind ausreichend groß. Das Fenster umfasst Takt 1 bis einschließlich des letzten Retire. Hier treten noch keine Fehler auf.

- **IO:** Start in Programmreihenfolge; nur der nächste noch nicht gestartete Befehl wird betrachtet. Seine Wartezeit hält jüngere Starts auf.
- **OB:** Out-of-Order mit blockierender LSU. Aus allen bereiten, noch nicht gestarteten Befehlen mit verfügbarer Einheit wird der älteste gewählt. Ein Load blockiert die LSU bis zum Ende seiner Fertigstellung; der nächste Load kann frühestens im Folgetakt starten.
- **ON:** Gleiche Auswahl wie OB, aber nichtblockierende LSU mit Initiationsintervall eins. Noch laufende Loads verhindern keinen neuen Loadstart; dessen Latenz bleibt fünf Takte. Die globale Startbreite bleibt eins.

1. Nennen Sie sämtliche RAW-Kanten sowie WAR und WAW zwischen diesen Befehlen. Welcher der beiden Loads liefert den Wert für B, welcher für D? (2 P)
2. Die Anfangsabbildung ist `s0 → P1`, `t0 → P5`, `t1 → P6`, `t2 → P7`, `t3 → P8`, `t4 → P9`, `t5 → P10`, `t6 → P11`. A bis F erhalten der Reihe nach die freien Ziele P20 bis P25. Erstellen Sie eine Tabelle mit physischen Quellen, neuem Ziel und bisheriger Zielabbildung. Bestimmen Sie die Endwerte von `t0`, `t1`, `t3`, `t5`, `t6`. (3 P)
3. Erstellen Sie je eine Tabelle mit Start, Fertigstellung und Retire von A bis F für IO und ON. Trennen Sie eine fehlende Wertbereitschaft von einer durch das geordnete Retire verursachten Wartezeit. (4 P)
4. Bestimmen Sie dieselben Zeiten für OB. Erläutern Sie, weshalb OoO allein hier wesentlich weniger gewinnt als zusammen mit der nichtblockierenden LSU. (2 P)

5. Beschreiben Sie für ON den ROB am Ende von Takt 4: Welche Operationen laufen, welche sind fertig, welche warten worauf, was ist bereits bestätigt? Bestimmen Sie ON-IPC und den Speedup gegenüber IO. Beurteilen Sie: „ON gewinnt, weil der einzelne Load schneller wird.“ (3 P)

Musterlösung 3

1. RAW: A→B auf `t0`, C→D auf `t0`, E→F auf `t5`, B→F auf `t1`. WAR: B→C auf `t0`. WAW: A→C auf `t0`. B liest die Version von A, D die Version von C. Es gibt keine RAW-Kante A→C: C benutzt den vorherigen Inhalt von `t0` nicht. **Bewertung:** RAW 1 P, WAR/WAW und Versionszuordnung 1 P.

Befehl	Physische Quellen	Neues Ziel	Bisheriges Ziel
A	P1	P20	P5
B	P20, P7	P21	P6
C	P1	P22	P20
D	P22, P9	P23	P8
E	P7, P9	P24	P10
F	P24, P21	P25	P11

2. Die Quellen werden bei der Umbenennung festgehalten. B liest weiter P20, obwohl die spekulative Abbildung von `t0` danach P22 heißt. Es gilt `t0 = 12`, `t1 = 23`, `t3 = 3`, `t5 = 3 XOR 9 = 10`, `t6 = 10 AND 23 = 2`. **Bewertung:** Quellen/Ziele 2 P, sämtliche Endwerte 1 P.

Befehl	IO: Start / fertig / Retire	ON: Start / fertig / Retire	OB: Start / fertig / Retire
A	1 / 5 / 6	1 / 5 / 6	1 / 5 / 6
B	6 / 6 / 7	6 / 6 / 7	6 / 6 / 7
C	7 / 11 / 12	2 / 6 / 7	7 / 11 / 12
D	12 / 12 / 13	7 / 7 / 8	12 / 12 / 13
E	13 / 13 / 14	3 / 3 / 8	2 / 2 / 13
F	14 / 14 / 15	8 / 8 / 9	8 / 8 / 14

3. IO wartet nach A auf B. ON kann zunächst C und E vorziehen. In Takt 6 ist B der älteste bereite Kandidat; D kann erst nach C in Takt 7 starten. ON-E ist schon Ende 3 fertig, darf aber erst in Takt 8 zusammen mit D retiren. F hat vor Ende 6 nicht beide Quellen; in Takt 7 erhält der ältere D den einzigen Startplatz. **Bewertung:** IO-Tabelle 1 P, ON-Tabelle 2 P, Bereitschaft und Retire-Warten unterscheiden 1 P.
4. Bei OB startet E in Takt 2. C darf wegen der blockierten LSU zunächst nicht starten; in Takt 6 erhält der ältere B Vorrang. C beginnt daher erst in Takt 7. In Takt 8 kann F starten, wartet dann aber hinter C, D und E auf Retire. OB benötigt 14 statt 15 Takte. Das Vorziehen von E verkürzt weder die beiden seriellen Loads noch deren Abhängigkeitsketten. ON überlappt zusätzlich die beiden Loads und endet schon in Takt 9. **Bewertung:** OB-Tabelle 1 P, Engpassvergleich 1 P.

5. Ende Takt 4 gilt für ON:

ROB-Eintrag	Zustand
A	Load läuft; Ergebnis erst Ende 5
B	wartet auf P20 von A
C	Load läuft; Ergebnis erst Ende 6
D	wartet auf P22 von C
E	fertig; P24 enthält 10, Retire durch ältere Einträge blockiert
F	P24 bereit, wartet auf P21 von B

Noch kein Befehl ist bestätigt; die architektonischen Register haben die Anfangswerte. $\mathrm{IPC}_{\mathrm{ON}} = 6/9 = 2/3$; $S_{\mathrm{ON/IO}} = 15/9 = 5/3 \approx 1{,}667$. Jeder Load behält fünf Takte Latenz. Der Gewinn stammt aus Überlappung und der Auswahl bereiter Operationen. **Bewertung:** ROB 2 P, Kennzahlen und Beurteilung 1 P.

Aufgabe 4: Ein besserer Prädiktor für jeden Code? (10 Punkte)

Papieraufgabe. Die Prädiktoren betreffen jeweils dieselbe einzelne Sprungstelle; Tabellen-Aliasing gibt es nicht. T bedeutet genommen, NT nicht genommen. Die Vorhersage benutzt immer den alten Zustand, erst anschließend wird mit dem tatsächlichen Ergebnis aktualisiert. Der Ein-Bit-Prädiktor speichert das letzte Ergebnis. Beim Zwei-Bit-Sättigungszähler sagen `00`, `01` NT und `10`, `11` T vorher; T erhöht, NT erniedrigt den Zähler, jeweils mit Sättigung.

1. Ein Programm liefert die acht Ergebnisse **T, T, NT, NT, T, T, NT, NT**. Der Ein-Bit-Prädiktor startet mit T, der Zwei-Bit-Prädiktor in `11`. Notieren Sie je Schritt die Ein-Bit-Vorhersage sowie alten Zustand und Vorhersage des Zwei-Bit-Prädiktors. Zählen Sie die Fehler. Widerlegt das Ergebnis die Behauptung „Zwei Bit sind immer besser“? Erklären Sie, welche Eigenschaft hier nachteilig wirkt. (3 P)

2. Für einen **anderen, langen** Befehlsstrom sind 15 % der bestätigten Befehle bedingte Sprünge. Als Eingabe gelten die eingeschwungenen Fehlraten $m_1 = 2/3$ für einen Ein-Bit- und $m_2 = 1/3$ für einen Zwei-Bit-Prädiktor. Fehlrate bedeutet hier Fehler je bedingtem Sprung. Alle Zusatzkosten der Fehlvorhersage sind in der folgenden Penalty enthalten; sonstige Effekte bleiben in diesem Rechenmodell konstant.

Entwurf	Taktfrequenz	Basis-CPI	Zusätzliche Penalty je Fehler
P	1 GHz	1,0	8 Takte
Q, tiefere Pipeline	1,5 GHz	1,2	16 Takte

   Bestimmen Sie für jede Kombination aus Entwurf und Prädiktor die gesamte CPI und Zeit je bestätigtem Befehl. Welcher Entwurf gewinnt jeweils? Bestimmen Sie außerdem die Fehlrate m, bei der P und Q gleich schnell sind. (4 P)

3. Q benötigt die 1,2-fache Spannung von P. Für diesen Teil gelten gleiche Schaltaktivität α und gleiche effektive Kapazität C_{eff}; statische Leistung wird vernachlässigt. Verwenden Sie in beiden Entwürfen den Zwei-Bit-Prädiktor aus Teil 2 und $P_{\text{dyn}} = \alpha C_{\text{eff}} V^2 f$. Vergleichen Sie die dynamische Leistung und Energie je bestätigtem Befehl. Begründen Sie, weshalb schneller hier nicht energiesparender heißt. Nennen Sie eine Grenze dieses idealisierten Vergleichs. (3 P)

Musterlösung 4

Schritt	Ist	1 Bit: Vorhersage	1 Bit: Fehler?	2 Bit: alt	2 Bit: Vorhersage	2 Bit: Fehler?
1	T	T	nein	`11`	T	nein
2	T	T	nein	`11`	T	nein
3	NT	T	ja	`11`	T	ja
4	NT	NT	nein	`10`	T	ja
5	T	NT	ja	`01`	NT	ja
6	T	T	nein	`10`	T	nein
7	NT	T	ja	`11`	T	ja
8	NT	NT	nein	`10`	T	ja

1. Ein Bit verursacht drei Fehler, zwei Bit fünf. Die Hysterese verhindert bei kurzen Richtungsblöcken einen rechtzeitigen Wechsel: Auf das erste NT folgt noch eine T-Vorhersage; nach dem zweiten NT folgt dann eine NT-Vorhersage, obwohl wieder T kommt. Mehr Zustand

ist eine Möglichkeit für eine andere Strategie, keine Garantie für jede Folge. **Bewertung:** Spuren/Fehlerzahlen 2 P, Gegenbeispiel und Begründung 1 P.

2. Es gelten $\mathrm{CPI} = \mathrm{CPI}_0 + bmp$ und $t_{\mathrm{Befehl}} = \mathrm{CPI}/f$ mit $b = 0{,}15$.

Entwurf / Prädiktor	Zusatz-CPI	Gesamt-CPI	Zeit je Befehl
P / 1 Bit	0,8	1,8	1,8 ns
Q / 1 Bit	1,6	2,8	1,867 ns
P / 2 Bit	0,4	1,4	1,4 ns
Q / 2 Bit	0,8	2,0	1,333 ns

Mit einem Bit benötigt Q etwa 3,7 % mehr Zeit als P; mit zwei Bit ist Q um den Faktor $1{,}4/(2/1{,}5) = 1{,}05$ schneller. Für eine gemeinsame Fehlrate m gilt beim Gleichstand

$$(1 + 1{,}2m)/1 = (1{,}2 + 2{,}4m)/1{,}5,$$

also $m = 0{,}5$. Q ist nur für $m < 0{,}5$ schneller. **Bewertung:** vollständige CPI-/Zeittabelle 2 P, Entwurfsvergleich 1 P, Schwelle und Richtung 1 P.

3. Die dynamische Leistung steigt um

$$\frac{P_{\mathrm{dyn},Q}}{P_{\mathrm{dyn},P}} = 1{,}2^2 \cdot 1{,}5 = 2{,}16.$$

Für dieselbe Zählbasis gilt

$$E_{\mathrm{Befehl}} = P_{\mathrm{dyn}} \frac{\mathrm{CPI}}{f} = \alpha C_{\mathrm{eff}} V^2 \mathrm{CPI}.$$

Daher beträgt der Energiefaktor

$$\frac{E_Q}{E_P} = 1{,}2^2 \frac{2{,}0}{1{,}4} \approx 2{,}057.$$

Der Takt kürzt sich explizit heraus; Spannung und Zahl der Takte pro bestätigtem Befehl bleiben. Q ist mit zwei Bit nur wenig schneller und braucht in diesem Modell gut doppelt so viel dynamische Energie je Befehl. Mögliche Grenzen: mehr Pipelineregister ändern die Kapazität, verworfene Pfade die Schaltaktivität, reale Speicherzugriffe die CPI; statische Leistung fehlt. **Bewertung:** Leistungsfaktor 1 P, Energie/Rechenweg 1 P, Erklärung und eine Grenze 1 P.

Aufgabe 5: Fertig ist noch nicht bestätigt (8 Punkte)

Papieraufgabe. Verwenden Sie Programm, physische Register und Entwurf ON von Aufgabe 3. Betrachten Sie zunächst dessen fehlerfreien Ablauf. Es gilt das einfache Freigabeverfahren der Vorlesung: Ein bisheriges physisches Ziel wird freigegeben, wenn der nachfolgende Überschreiber desselben Architekturregisters erfolgreich retiert. Die bestätigte Abbildung ist von der spekulativen Rename-Abbildung getrennt; beim Trap wird die bestätigte Abbildung wiederhergestellt. Andere Referenzen wie Checkpoints werden in diesem Lehrmodell nicht zusätzlich gehalten.

1. Darf P20 schon beim Rename von C, bei der Fertigstellung von A oder bei der Fertigstellung von C freigegeben werden? Nennen Sie den tatsächlichen Freigabezeitpunkt und begründen Sie ihn mit Leser und Wiederherstellung. (2 P)
2. Ändern Sie jetzt **nur** C: Statt eines Werts meldet der Load am Ende von Takt 6 eine Zugriffs-Exception. Ein fehlerhafter Befehl retiert nicht. Beim Retire wird zuerst der fertige ältere Programmanfang bestätigt; erreicht die Exception den ROB-Kopf, wird sofort der präzise Trap ausgelöst und es startet in diesem Takt kein neuer Befehl. Wann tritt der Trap ein? Geben Sie die sichtbaren Werte von `t0`, `t1`, `t3`, `t5`, `t6`, die wiederhergestellte bestätigte Registerabbildung und die Behandlung der Ziele P22 bis P25 an. Darf P20 hier freigegeben werden? (3 P)
3. In einem **getrennten** Spekulationsfall wird ein bedingter Pfad später verworfen. Ein geheimer Bytewert $s \in \{0, \ldots, 255\}$ kann vorübergehend intern geladen werden. Entwerfen Sie als Pseudocode einen nachfolgenden Probezugriff, dessen Cache-Zeile vom Wert s abhängt. Lehrannahmen: 64-Byte-Cache-Zeilen, 256 gültige Probezeilen, anfangs keine davon im Cache, und kein Rückgängigmachen dieser Cache-Belegung beim Verwerfen. Erklären Sie, was trotz präziser Registerwiederherstellung beobachtbar bleiben kann. Warum beweist ein einzelner schneller Zugriff noch keinen Geheimnisabfluss? (3 P)

Musterlösung 5

1. P20 enthält die von A erzeugte `t0`-Version. B muss genau diese Version lesen. Darüber hinaus ist P20 nach dem Retire von A die bestätigte Abbildung von `t0`; vor erfolgreichem Retire von C könnte eine Wiederherstellung sie noch benötigen. Weder Rename von C noch die Fertigstellung eines Loads genügen. Erst C gibt beim fehlerfreien Retire in Takt 7 sein bisheriges Ziel P20 frei; B ist im selben Takt vorher retiert. **Bewertung:** Zeitpunkt 1 P, Leser und Wiederherstellung 1 P.

2. A retiert in Takt 6. B wird Ende 6 fertig und retiert zu Beginn von Takt 7. Danach liegt die Exception von C am Kopf; der Trap tritt noch in Takt 7, vor einer neuen Startauswahl, ein. C und alle jüngeren Befehle werden verworfen. F startet in diesem Fehlerfall deshalb nicht mehr.

Architekturregister	Sichtbarer Wert	Bestätigte physische Abbildung
`s0`	unverändert	P1
`t0`	20	P20
`t1`	23	P21
`t2`	3	P7
`t3`	70	P8
`t4`	9	P9
`t5`	71	P10
`t6`	72	P11

Insbesondere ist der intern von E erzeugte Wert 10 nicht bestätigt. P22–P25 gehören zu verworfenen Zielen und werden wieder verfügbar; die spekulative Abbildung wird auf die Tabelle zurückgesetzt. P20 bleibt erhalten, weil C seine Überschreibung nie bestätigt hat. P5 und P6 wurden bereits durch A beziehungsweise B beim Retire freigegeben. **Bewertung:** Trapzeitpunkt 1 P, Werte/Abbildung 1 P, Freigabe und Verwerfen 1 P.

3. Ein möglicher Pseudocode mit 64-Byte-ausgerichtetem Probeanfang ist:

```
s = transient_load(secret_address)   // Lehrannahme: Byte intern verfügbar
x = transient_load(probe_base + 64*s)
```

Die zweite Adresse hängt vom geheimen **Wert** ab. Sie wählt genau eine von 256 unterschiedlichen Cache-Zeilen. Später kann man im Lehrmodell die Zugriffszeiten der Probezeilen vergleichen und daraus eine Hypothese über s ableiten. Das Zurücksetzen der Registerabbildung entfernt diese interne Cache-Spur gemäß der gegebenen Annahme nicht. Ein bloßer Load einer festen geheimen Adresse würde dagegen noch keine Probezeile abhängig vom gelesenen Wert auswählen.

Ein einzelner schneller Zugriff kann auch aus einem anderen Cache-Effekt, Vorladen oder Messrauschen entstehen. Ein Nachweis braucht reproduzierbare wertabhängige Unterschiede gegenüber kontrollierten Vergleichsfällen. Die Aufgabe beschreibt einen bedingten Lehrablauf, keine Behauptung über die Verwundbarkeit eines konkreten Prozessors. **Bewertung:** Wertkopplung/verschiedene Zeilen 1 P, verbleibende Spur 1 P, Nachweisgrenze 1 P.

Aufgabe 6: Welchen Engpass beseitigt der Compiler? (10 Punkte)

Papieraufgabe, M1 mit Forwarding. RV32I mit fünf Stufen IF, ID, EX, MEM, WB, je ein Takt; höchstens ein neuer Befehl je Takt. Ein ALU-Ergebnis kann an EX des direkt folgenden Befehls weitergereicht werden. Ein Loadwert liegt erst am Ende von MEM vor: Sein direkt folgender Nutzer wartet einen Takt in ID; jüngere Befehle warten mit. Mit einem unabhängigen Befehl dazwischen entsteht kein Load-Use-Wartetakt. Branch-Vergleiche verwenden ebenfalls Forwarding; Sprünge sind stets richtig vorhergesagt und erzeugen keine Zusatzzyklen. Das Fenster reicht vom ersten IF bis zum letzten WB des betrachteten Codes. Initialisierung und Rücksprung zur aufrufenden Funktion werden nicht mitgezählt.

Ein Compiler übersetzt eine Summe über 100 gültige vorzeichenlose 32-Bit-Werte. Die Speicherzugriffe sind ausgerichtet und ohne zusätzliche Wartezeit. Anfangs zeigt `a0` auf den ersten Wert, `a1` genau 400 Byte weiter; `t0 = 0`. Die Adressrechnung läuft nicht über. Alle Summen sind modulo 2^{32} zu verstehen.

```
loop: lw   t1, 0(a0)
      add  t0, t0, t1
      addi a0, a0, 4
      bne  a0, a1, loop
```

1. Bestimmen Sie den Engpass, die eingeschwungene Taktzahl je Element sowie Fenster und CPI für 100 Elemente. Ordnen Sie **dieselben vier Befehle** so um, dass der Wartetakt entfällt. Begründen Sie, warum jeder Load weiterhin die richtige Adresse liest. Geben Sie das neue Fenster und den Speedup an. (3 P)

2. Stattdessen wird zweifach ausgerollt und `t2` zusätzlich mit null initialisiert:

   ```
   loop: lw   t1, 0(a0)
         lw   t3, 4(a0)
         addi a0, a0, 8
         add  t0, t0, t1
         add  t2, t2, t3
         bne  a0, a1, loop
         add  t0, t0, t2     # erst nach der Schleife
   ```

   Bestimmen Sie Takte je Element im eingeschwungenen Zustand, dynamische Befehlszahl und gesamtes Fenster. Würde der Ersatz der zweiten Addition durch `add t0, t0, t3` mit nur einem Akkumulator wieder Wartetakte erzeugen? Bei dieser Ein-Akkumulator-Variante entfällt die Schlussaddition. Trennen Sie den Nutzen des Ausrollens vom Nutzen zusätzlicher Wertketten. (2 P)

3. Wechseln Sie zu einem anderen Lehrkern: Zwei vollständig gepipelinete Addierer können insgesamt zwei Additionen je Takt starten, jeder mit Latenz drei und Initiationsintervall eins. Eingabewerte stehen bereit; Loads, Schlussreduktion und Schleifenverwaltung begrenzen den eingeschwungenen Durchsatz nicht. Die Elemente werden gleichmäßig auf k unabhängige Akkumulatoren verteilt. Bestimmen Sie die minimalen mittleren Takte je Element für $k = 1, 2, 3, 6, 8$. Wie viele Akkumulatoren sind mindestens nötig, um die Addierer auszulasten? Erklären Sie den Unterschied zu Teil 2. (2 P)

4. Prüfen Sie die Umformung auf der Ebene von C17, getrennt vom Zeitmodell. Warum darf eine reine `uint32_t`-Summe modulo 2^{32} in Teilketten aufgeteilt werden? Vergleichen Sie damit die Summe über `float a[3] = {1e8f, -1e8f, 1.0f}`: einmal von links nach rechts mit Startwert null, einmal mit getrennten Akkumulatoren für gerade und ungerade Indizes, anschließend zusammengeführt. Für dieses Beispiel gilt IEEE-754 binary32 mit Rundung nach jeder Addition auf den nächsten Wert, bei Gleichstand zum geraden Wert. 10^8 ist genau darstellbar; benachbarte Werte liegen dort um acht auseinander. Zusätzliche Genauigkeit und Umklammerung sind ausgeschlossen. Welche Ergebniszusage müsste eine Optimierung hier beachten? Darf außerdem ein Load der nächsten Iteration über einen neu ergänzten Store einer Zwischensumme vorgezogen werden, nur weil er ein anderes Basisregister verwendet? Nennen Sie die benötigte Zusatzinformation. (3 P)

Musterlösung 6

1. Der RAW-Konflikt vom Load zur direkt folgenden Addition verursacht pro Element einen Wartetakt. Es werden 400 Befehle ausgeführt; Füllen und Leeren ergeben zusammen vier zusätzliche Takte. Damit folgen eingeschwungen fünf Takte je Element, insgesamt $400 + 100 + 4 = 504$ Takte und $\mathrm{CPI} = 504/400 = 1{,}26$. Eine zulässige Umordnung lautet:

```
loop: lw   t1, 0(a0)
      addi a0, a0, 4
      add  t0, t0, t1
      bne  a0, a1, loop
```

`lw` verwendet weiterhin die alte Adresse vor `addi`; der WAR-Konflikt auf `a0` wird also nicht verletzt. Die Addition liest den Loadwert erst mit einem Befehlsabstand; der Sprung vergleicht wie zuvor das erhöhte `a0`. Es entstehen vier Takte je Element, Fenster $400 + 4 = 404$ und $\mathrm{CPI} = 404/400 = 1{,}01$; $S = 504/404 \approx 1{,}248$. **Bewertung:** ursprüngliche Kennzahlen/Engpass 1 P, Umordnung mit Adressbegründung 1 P, neues Fenster/Speedup 1 P.

2. Beide Loads werden hinreichend spät genutzt, daher keine Wartetakte. Sechs Schleifenbefehle bearbeiten zwei Elemente: eingeschwungen drei Takte je Element. Insgesamt $50 \cdot 6 + 1 = 301$ Befehle und $301 + 4 = 305$ Takte. Gegenüber dem umgeordneten Code ist das ein Speedup von $404/305 \approx 1{,}325$.

   Mit nur einem Akkumulator entsteht in M1 ebenfalls kein Wartetakt: Die beiden aufeinanderfolgenden Additionen können das ALU-Ergebnis per Forwarding nutzen. Dann sind es 300 Befehle und 304 Takte, weil keine Schlussaddition nötig ist. Die Startfolge innerhalb der Schleife ist gleich. Der Durchsatzgewinn stammt hier vom eingesparten `addi` und `bne` pro zwei Elemente, nicht von zwei Akkumulatoren. **Bewertung:** zwei Akkumulatoren mit Zählung 1 P, Ein-Akkumulator-Vergleich/Begründung 1 P.

3. Eine einzelne Kette kann nur alle drei Takte weitergeführt werden. k Ketten bieten höchstens $k/3$ Additionen je Takt, die Hardware höchstens zwei. Somit gilt für den minimalen mittleren Taktbedarf

$$c(k) = \max\left(\frac{3}{k}, \frac{1}{2}\right).$$

Akkumulatoren k	1	2	3	6	8
Takte je Element	3	1,5	1	0,5	0,5

Sechs Ketten reichen: In drei Takten werden jeweils zwei andere Ketten bedient, danach sind die ersten Ergebnisse nutzbar. Weitere Ketten erhöhen die Startkapazität nicht. In Teil 2 hatte die Addierkette dagegen Latenz eins bei nur einem möglichen Start je Takt; ein Akkumulator genügte bereits. **Bewertung:** Formel/Tabelle 1 P, Mindestzahl und Modellunterschied 1 P.

4. Vorzeichenlose Addition modulo 2^{32} ist assoziativ und kommutativ. Ein Aufteilen mit anschließender Reduktion erhält daher den vereinbarten `uint32_t`-Rückgabewert, sofern gültige Elemente jeweils genau einmal berücksichtigt werden. Dies ist keine entsprechende Zusage für vorzeichenbehafteten Integerüberlauf.

   Bei binary32 liefert links-nach-rechts zuerst $10^8 + (-10^8) = 0$ und danach $0 + 1 = 1$. Die gerade Teilkette berechnet dagegen $10^8 + 1$, das auf 10^8 gerundet wird; die ungerade ist -10^8. Die Schlussaddition liefert null. Eine Optimierung muss den vereinbarten Gleitkomma-Ergebnisvertrag erhalten; ohne Erlaubnis für eine veränderte Rundung ist die Umklammerung nicht allgemein zulässig. Die bloße Wahl einer hohen Optimierungsstufe ist kein Nachweis mathematischer Assoziativität von `float`.

   Verschiedene Basisregister beweisen keine verschiedenen Speicherbereiche. Der vorgezogene Load könnte einen Wert lesen, den der Store erst ändern soll. Die Umordnung erfordert nachgewiesene Unabhängigkeit der betroffenen Speicherzugriffe, etwa eine gültige Nichtüberlappungszusage; ein `restrict`-Vertrag darf nur verwendet werden, wenn seine Voraussetzungen tatsächlich eingehalten werden. **Bewertung:** unsigned-Begründung 1 P, beide float-Ergebnisse und Vertrag 1 P, Aliasproblem/Zusicherung 1 P.

Optionale Vertiefung: Drei Teilketten funktional prüfen

Unbewertet; zusätzlich zum Papierumfang. Ergänzen Sie in `src/lab07_ilp.c` eine dritte Variante `sum3` mit drei `uint64_t`-Akkumulatoren, je einer für jede Position innerhalb einer Dreiergruppe. Verarbeiten Sie Restwerte anschließend ohne Zugriff außerhalb des Feldes. Verwenden Sie C17; Eingaben sind `uint32_t`, das Ergebnis eine Summe modulo 2^{64}. Bei Länge null darf kein Arrayelement gelesen werden.

Prüfen Sie die drei Summenvarianten für `{9,1,8,2,7,3,6}`, `{4,10}` und die leere Eingabe. Bestimmen Sie vorab die drei Teilsummen des ersten Falls vor und nach der Restbehandlung. Untersuchen Sie optional die Übersetzung mit einer hohen Optimierungsstufe. Halten Sie funktionale Übereinstimmung, erzeugte Maschinenbefehle und eine tatsächliche Laufzeitmessung als getrennte Nachweise auseinander; dokumentieren Sie bei einem Versuch Compiler, Version und Optionen. Die vorhandene Datei bleibt Startpunkt, es ist kein zusätzliches Projekt nötig.

Musterlösung der optionalen Vertiefung

Eine mögliche Ergänzung zur vorhandenen Datei ist:

```
static uint64_t sum3(const uint32_t *a, size_t n)
{
    uint64_t s0 = 0, s1 = 0, s2 = 0;
    size_t groups = n / 3;
    for (size_t j = 0; j < groups; ++j) {
        s0 += a[3*j];
        s1 += a[3*j + 1];
        s2 += a[3*j + 2];
    }
    for (size_t i = 3*groups; i < n; ++i)
        s0 += a[i];
    return s0 + s1 + s2;
}
```

Für `{9,1,8,2,7,3,6}` sind die Teilstände nach den beiden vollständigen Gruppen 11, 8 und 11. Der Restwert 6 erhöht die erste Summe auf 17; das Ergebnis ist 36. Für `{4,10}` entstehen keine vollständigen Gruppen; die Restschleife liefert 14. Bei Länge null wird keine Schleife betreten, auch mit einem Nullzeiger ist das Ergebnis dann null. Für positive Länge muss das Array die angegebenen gültigen Elemente bereitstellen. Da $3\lfloor n/3 \rfloor \leq n$ gilt, bleiben Gruppen- und Restindizes innerhalb dieses Vertrags; die Multiplikationen mit drei laufen für die tatsächlich verwendeten Indizes nicht über.

Die drei Additionsketten sind innerhalb der Gruppenschleife unabhängig; die Schlussreduktion verbindet sie wieder. Funktionstests beweisen keine kürzere Laufzeit. Auch eine andere Assemblerfolge ist erst ein Befund zur Übersetzung; Cachezustand, Rechner und Messmethode wären für einen Zeitvergleich zusätzlich festzulegen. Hier werden keine Host-Messwerte behauptet.